

Accuracy Analysis of an Approximation of the Gini Shortfall

Samuel Reano

Project overview

This project showcases an accuracy analysis on one approximation formula of the Gini Shortfall (a risk measure) for the truncated Normal Distribution and the bounded Pareto distribution. This is an excerpt of chapter 3 from my master’s thesis.

Introduction to the Gini Shortfall

The Gini Shortfall is a risk measure that is mathematically expressed as

$$\text{GS}_\beta^\lambda(X) = \frac{1}{(1-\beta)^2} \int_\beta^1 \text{VaR}_u(X) \left(1 - \beta + 4\lambda \left(u - \frac{(1+\beta)}{2}\right)\right) du$$

where $\text{VaR}_u(X)$ is Value-at-Risk, $0 < \beta < 1$ is the prudence level, and $\lambda \geq 0$ is the risk manager’s aversion level with respect to tail dispersion. If the risk manager is rational, then $0 \leq \lambda \leq \frac{1}{2}$. This is one of the many risk measures that a risk-averse risk manager would use. The Gini Shortfall provides information on the average severity and the variability of a loss distribution tail. That is, the Gini Shortfall focuses on the expected risk level and the variability of tail risk beyond an extreme quantile (i.e., beyond the Value-at-Risk). To see this, the Gini Shortfall can be rewritten as

$$\begin{aligned} \text{GS}_\beta^\lambda(X) &= \frac{1}{1-\beta} \int_\beta^1 \text{VaR}_u(X) du + \lambda \left(\frac{2}{(1-\beta)^2} \int_\beta^1 (2u - (1+\beta)) \text{VaR}_u(X) du \right) \\ \text{GS}_\beta^\lambda(X) &= \text{ES}_\beta(X) + \lambda \text{TGini}_\beta(X) \end{aligned}$$

To compute the Gini Shortfall in a programming language like R, one approach is to use approximation methods. One approximation method we present combines probability theory and numerical methods and it is capable only in bounded continuous distributions. The approximated Gini Shortfall is based on this scheme is mathematically expressed as

$$\text{GS}_\beta^\lambda(Y_N) = \int_{F_X^{-1}\left(\frac{\lceil N\beta \rceil - 1}{N}\right)}^{F_X^{-1}\left(\frac{\lceil N\beta \rceil}{N}\right)} x f_X(x) dx \left[\frac{\lceil N\beta \rceil - N\beta}{1-\beta} + \frac{2\lambda}{(1-\beta)^2} \left[\frac{\lceil N\beta \rceil^2}{N} - N\beta^2 + (1+\beta)(N\beta - \lceil N\beta \rceil) \right] \right] + \sum_{k=\lceil N\beta \rceil + 1}^N \int_{F_X^{-1}\left(\frac{\lceil N\beta \rceil - 1}{N}\right)}^{F_X^{-1}\left(\frac{\lceil N\beta \rceil}{N}\right)} x f_X(x) dx \left(\frac{1}{(1-\beta)} + \frac{2\lambda}{(1-\beta)^2} \left(\frac{2k-1}{N} - (1+\beta) \right) \right)$$

where N is the number of partitions, $f_X(x)$ is the explicit probability density function, and F_X^{-1} is the quantile function. Since some continuous distributions don’t have an explicit or analytical form of the quantile function, we will use the explicit cumulative distribution function and perform numerical inversion via Bisection method to calculate the bounds of the integration.

Truncated Normal Distribution $\mathcal{N}(2, 25)$

The following R code chunk computes the true Gini Shortfall. We truncate the support of the normal distribution from $(-\infty, \infty)$ to $[-10^4, 10^4]$.

Hide

```
library(distributionsrd) # Required for the truncated quantile function

# Parameters of GS
beta <- 0.01
lambda <- 0.25

# Truncated Normal Quantile Function (VaR)
VaR_norm <- function(x) {
  qtruncdist(x, dist = "norm",
             coeff = list(mean = 2, sd = 5),
             lowertrunc = -10000, uppertrunc = 10000)
}

# Expected Shortfall (ES)
ES_norm <- 1 / (1 - beta) * integrate(function(x) VaR_norm(x), lower = beta, upper = 1)$value

# Tail Gini calculation
integral_for_TG_norm <- integrate(function(x) VaR_norm(x) * (2 * x - (1 + beta)), lower = beta, upper = 1)$value
tail_gini_norm <- (2 / (1 - beta)^2) * integral_for_TG_norm

# Exact Gini Shortfall
exact_GS <- ES_norm + lambda * tail_gini_norm
```

Here, we present the algorithms for computing the approximated Gini Shortfall.

Hide

```

# Inverse CDF via Bisection Root-Finding
inverse_cdf <- function(F, u, support_left, support_right, tol = 1e-10, max_iter = 200) {
  lower <- support_left
  upper <- support_right

  for (i in 1:max_iter) {
    mid <- (lower + upper) / 2
    F_mid <- F(mid)

    if (abs(upper - lower) < tol) {
      return(mid)
    }

    if (F_mid < u) {
      lower <- mid
    } else {
      upper <- mid
    }
  }

  warning("Max iterations reached in bisection")
  return((lower + upper) / 2)
}

# Split Spectral Risk Measure Computation (Error bounds omitted)
compute_srm <- function(F, f, N, beta, lambda, support_left, support_right, tol = 1e-10) {
  k_star <- ceiling(N * beta)
  total_srm <- 0

  # Boundary Bin (k = k*)
  u_lower_star <- (k_star - 1) / N
  u_upper_star <- k_star / N

  x_lower_star <- inverse_cdf(F, u_lower_star, support_left, support_right, tol = tol)
  x_upper_star <- inverse_cdf(F, u_upper_star, support_left, support_right, tol = tol)

  res_star <- integrate(function(x) x * f(x), lower = x_lower_star, upper = x_upper_star, rel.tol = 1e-10)

  w_star <- ((k_star - N * beta) / (1 - beta)) +
    ((2 * lambda) / (1 - beta)^2) * ((k_star^2 / N) - beta^2 * N + (1 + beta) * (N * beta - k_star))

  total_srm <- total_srm + (res_star$value * w_star)

  # Remaining Bins (k = k* + 1 to N)
  if (k_star < N) {
    for (k in (k_star + 1):N) {
      u_lower <- (k - 1) / N
      u_upper <- k / N

      x_lower <- inverse_cdf(F, u_lower, support_left, support_right, tol = tol)
      x_upper <- inverse_cdf(F, u_upper, support_left, support_right, tol = tol)

      res_k <- integrate(function(x) x * f(x), lower = x_lower, upper = x_upper, rel.tol = 1e-10)

      w_k <- (1 / (1 - beta)) +
        ((2 * lambda) / (1 - beta)^2) * (((2 * k - 1) / N) - (1 + beta))

      total_srm <- total_srm + (res_k$value * w_k)
    }
  }

  return(total_srm)
}

```

The accuracy results:

Hide

```
library(distributionsrd) # Required for the truncated normal distribution

N <- 100000

# Truncated Normal Cumulative Distribution & Density Functions
F_X <- function(x) ptruncdist(x, dist = "norm", coeff = list(mean = 2, sd = 5), lowertrunc = -10000, uppertrunc = 10000)
f_X <- function(x) dtruncdist(x, dist = "norm", coeff = list(mean = 2, sd = 5), lowertrunc = -10000, uppertrunc = 10000)

# Compute Approximate Gini Shortfall
approx_GS <- compute_srm(F_X, f_X, N, beta = alpha, lambda = lambda, support_left = -10000, support_right = 10000)

# Calculate Absolute Error
abs_error <- abs(exact_GS - approx_GS)

# Display Output
results <- data.frame(
  Metric = c("Exact GS", "Approximate GS", "Absolute Error"),
  Value = c(exact_GS, approx_GS, abs_error)
)

print(results)
```

	Metric	Value
1	Exact GS	3.505695e+00
2	Approximate GS	3.505695e+00
3	Absolute Error	2.412594e-08

Bounded Pareto Distribution Pareto(1,1.1)

The following R code chunk computes the true Gini Shortfall. We truncate the support of the Pareto type I distribution from $[1, \infty)$ to $[1, 10^4]$.

Hide

```
library(ReIns) # Required for the truncated quantile function

# Parameters of GS
beta <- 0.01
lambda <- 0.25

# Truncated Pareto Quantile Function (VaR)
VaR_pareto <- function(x) qtpareto(x, shape = 1.1, scale = 1, endpoint = 10000)

# Expected Shortfall (ES)
ES_pareto <- 1 / (1 - beta) * integrate(function(x) VaR_pareto(x), lower = beta, upper = 1)$value

# Tail Gini calculation
integral_for_TG_pareto <- integrate(function(x) VaR_pareto(x) * (2 * x - (1 + beta)), lower = beta, upper = 1)$value
tail_gini_pareto <- (2 / (1 - beta)^2) * integral_for_TG_pareto

# Exact Gini Shortfall
exact_GS <- ES_pareto + lambda * tail_gini_pareto
```

Here, we present the algorithms for computing the approximated Gini Shortfall.

Hide

```

# Inverse CDF via Bisection Root-Finding
inverse_cdf <- function(F, u, support_left, support_right, tol = 1e-10, max_iter = 200) {
  lower <- support_left
  upper <- support_right

  for (i in 1:max_iter) {
    mid <- (lower + upper) / 2
    F_mid <- F(mid)

    if (abs(upper - lower) < tol) {
      return(mid)
    }

    if (F_mid < u) {
      lower <- mid
    } else {
      upper <- mid
    }
  }

  warning("Max iterations reached in bisection")
  return((lower + upper) / 2)
}

# Split Spectral Risk Measure Computation (Error bounds omitted)
compute_srm <- function(F, f, N, beta, lambda, support_left, support_right, tol = 1e-10) {
  k_star <- ceiling(N * beta)
  total_srm <- 0

  # Boundary Bin (k = k*)
  u_lower_star <- (k_star - 1) / N
  u_upper_star <- k_star / N

  x_lower_star <- inverse_cdf(F, u_lower_star, support_left, support_right, tol = tol)
  x_upper_star <- inverse_cdf(F, u_upper_star, support_left, support_right, tol = tol)

  res_star <- integrate(function(x) x * f(x), lower = x_lower_star, upper = x_upper_star, rel.tol = 1e-10)

  w_star <- ((k_star - N * beta) / (1 - beta)) +
    ((2 * lambda) / (1 - beta)^2) * ((k_star^2 / N) - beta^2 * N + (1 + beta) * (N * beta - k_star))

  total_srm <- total_srm + (res_star$value * w_star)

  # Remaining Bins (k = k* + 1 to N)
  if (k_star < N) {
    for (k in (k_star + 1):N) {
      u_lower <- (k - 1) / N
      u_upper <- k / N

      x_lower <- inverse_cdf(F, u_lower, support_left, support_right, tol = tol)
      x_upper <- inverse_cdf(F, u_upper, support_left, support_right, tol = tol)

      res_k <- integrate(function(x) x * f(x), lower = x_lower, upper = x_upper, rel.tol = 1e-10)

      w_k <- (1 / (1 - beta)) +
        ((2 * lambda) / (1 - beta)^2) * (((2 * k - 1) / N) - (1 + beta))

      total_srm <- total_srm + (res_k$value * w_k)
    }
  }

  return(total_srm)
}

```

The accuracy results:

Hide

```
library(ReIns) # Required for the bounded Pareto distribution

N <- 100000

# Bounded Pareto Cumulative Distribution & Density Functions
F_X <- function(x) ptpareto(x, shape = 1.1, scale = 1, endpoint = 100000)
f_X <- function(x) dtpareto(x, shape = 1.1, scale = 1, endpoint = 100000)

# Compute Approximate Gini Shortfall
approx_GS <- compute_srm(F_X, f_X, N, beta = alpha, lambda = lambda, support_left = 1, support_right = 10000)

# Calculate Absolute Error
abs_error <- abs(exact_GS - approx_GS)

# Display Output
results <- data.frame(
  Metric = c("Exact GS", "Approximate GS", "Absolute Error"),
  Value = c(exact_GS, approx_GS, abs_error)
)

print(results)
```

	Metric	Value
1	Exact GS	9.0918546908
2	Approximate GS	9.0913080287
3	Absolute Error	0.0005466621

Conclusion

The absolute errors for the truncated Normal distribution and the Bounded Pareto distribution are small and appealing to the practitioner. We deduce that the absolute errors can be smaller when increasing the number of partitions and extending the endpoints of the truncated support.